

1. Meeting Information

Date & Time:	03/25/2026	6:30-7:30 PM (60 Minutes)
Location:	Webex	
Date & Time:	04/01/2026	6:00-7:30 PM (90 Minutes)
Location:	Webex	
Date & Time:	04/08/2026	6:00-7:30 PM (90 Minutes)
Location:	Webex	

2. Meeting Overview

03/25

- Discussed component progress over the past 2 weeks, and any current challenges.
- Updated Miro Kanban board with new user stories and moved completed/backlogged items.
- Assigned team to-do's, discussed individual module 6 project, and 'music event coordinator' capstone project idea.

04/01

- Reviewed and approved Ricky's Dark mode PR, and applicable changes to other features.
- Assigned Dewey as team lead for upcoming 04/08 meeting, Lauren will be present until ~6:20.
- Assigned new component teams for 5/6 members, which are to be started once their main components are completed. An updated list is included in section 3. App Contributor Roles & Teams.
- Reviewed Carelink progress, which has a **final deadline of Sunday, 04/05 EOD**.

04/08

- Review open PRs. Ensure ENTIRE team is set up with the new PR titled: 'Frontend/Backend Integration' and is fully functional for everyone BEFORE starting secondary feature development. New branches should be started for secondary features.
 - Assign Dayne a secondary feature/team.
 - Complete all secondary feature initial sketches and Figma mockups by end of meeting.
 - **Secondary Feature Coding Deadline: 04/25/26 11:00 PM**
-

3. App Contributor Roles & Teams

<i>Name</i>	<i>Role</i>	<i>Functional & Component Sub-Team</i>
Lauren Knotts	Team Lead, Full-stack Dev., Scribe	Education System, Landing Page, User Connections
Ricardo Serrano	Full-stack Dev., Scribe	Authentication, Landing Page, Admin Permissions
Dewey Mccrary	Full-stack Dev.	Calendar Functionality, Notifications
Baudelaire Messago Ngumouo	Full-stack Dev.	User Connections, User Feedback
Tamia Jenkins	Full-stack Dev.	User Connections, Support FAQ
Dayne Allen-Wright	Full-stack Dev.	Calendar Functionality, Support FAQ

4. Team To-Do's

All team members are responsible for completing the following tasks:

- Submit design artifacts demonstrating how the Angular frontend and Spring Boot backend are packaged together into a single executable application using Maven.
 - Should show how the Angular production build is generated and copied into the Spring Boot project.
 - Includes sketches, diagrams, pictures, or flowcharts demonstrating the build process and project structure.

5. Decisions

- Team deadlines for submissions making it into the milestone document is **04/10/26 11:00 PM**
- Course Project Milestone Report document will be submitted no later than **04/11/26 12:00 PM**

6. Original Components Fully Integrated – Final Week

Component	Responsible Members	Completion Status
User Authentication (Login/Register/Logout)	Ricky	Complete
User Management (Carelink)	Baudelaire, Tamia	Complete
Appointment/Calendar	Dayne, Dewey	In Progress (Error Handling)
Education System	Lauren	Complete
Navigation Bar & Footer	Lauren	Complete
Main Dashboard	Ricky	Complete

Carelink Contributions:

- **Baudelaire**
 - **Contribution Timeline: February 19-20**
 - Created initial Carelink component, service, and model files.

- Initial CSS and UI files.
 - **Tamia**
 - **Contribution Timeline: February 25-March 25**
 - Frontend logic development.
 - UI/UX updates and styling refinements.
 - Integration testing for User IDs and roles.
 - **Lauren**
 - **Contribution Timeline: April 5-8**
 - Frontend-to-Backend integration and API connection fixes.
 - Complete rewrite of `carelink.component.ts` and `carelink.service.ts` to replace hardcoded data with dynamic API responses.
 - Database/Backend modifications to ensure data persistence for dated connections ('Connected Since' errors).
 - Updated API and UI logic to support role-based views and actions (Senior v. Caregiver).
 - EX: Only seniors can create/break connections, caregivers can only break them.
-

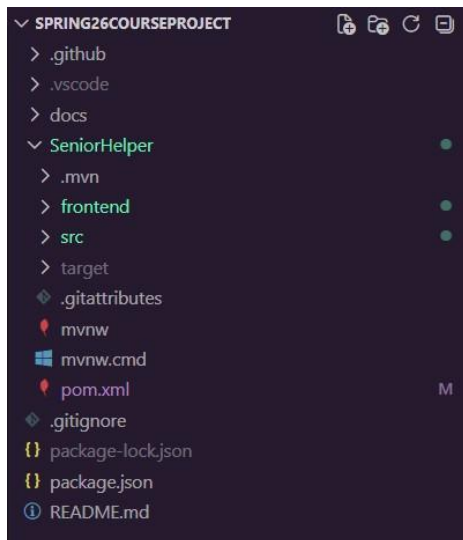
7. Frontend-Backend Integration

Overview:

The end goal was to package Angular + Sprint Boot into one executable Maven-built application. In other words, instead of having to run the backend through IntelliJ, and the front end through VSCode in order to access the app, now it's all bundled in one place, and only requires running the JAR file that gets generated.

Integration Steps:

Step 1. Cut and paste the entire SeniorHelperFrontEnd folder into the root SeniorHelper folder and rename it to "frontend".

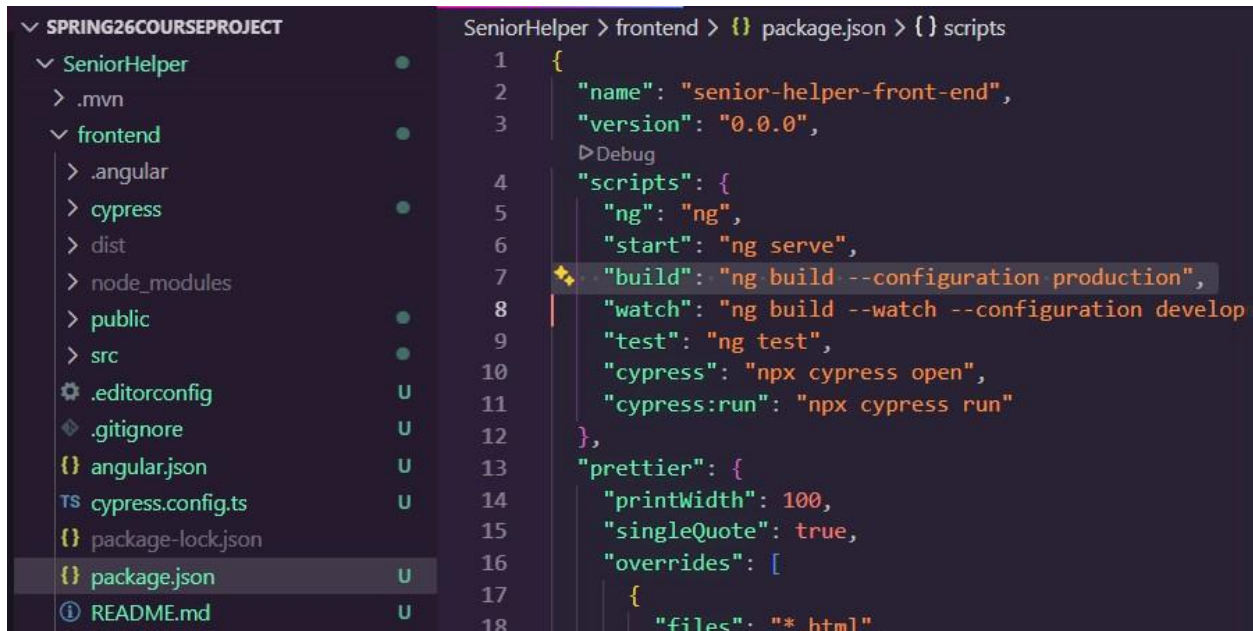


Step 2.

Update the Angular build configuration by opening the Angular project's package.json file inside SeniorHelper/frontend and changing the build script to:

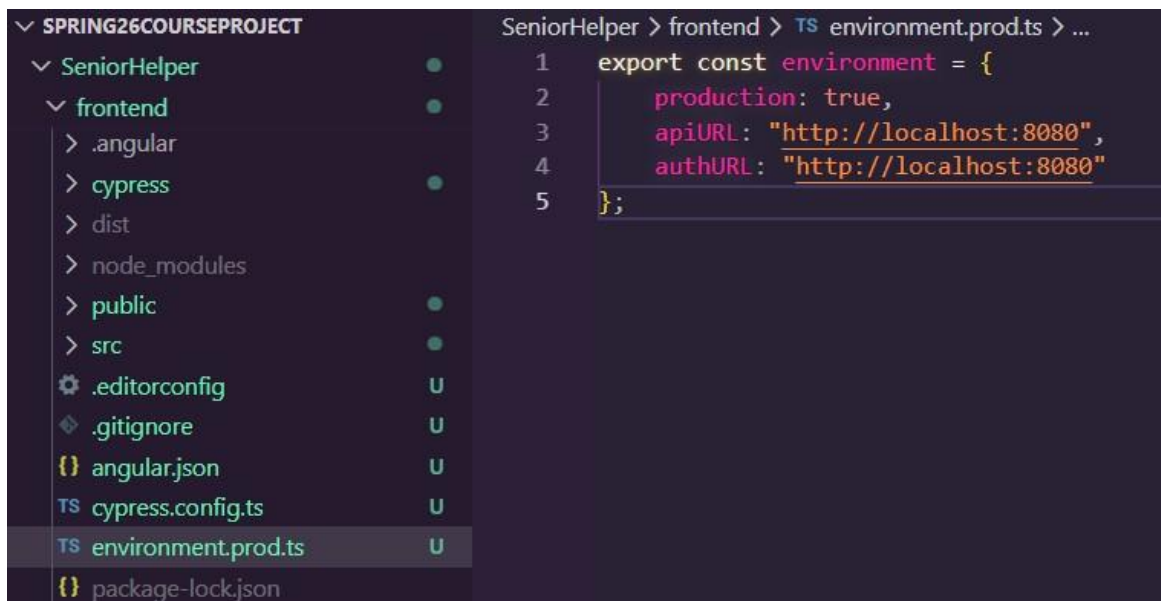
```
"build": "ng build --configuration production"
```

This ensures that when Maven runs the Angular build, it creates a production build.



```
SPRING26COURSEPROJECT
└─ SeniorHelper
   └─ frontend
      ├── .angular
      ├── cypress
      ├── dist
      ├── node_modules
      ├── public
      ├── src
      ├── .editorconfig
      ├── .gitignore
      ├── angular.json
      ├── cypress.config.ts
      ├── package-lock.json
      └── package.json
         {
           "name": "senior-helper-front-end",
           "version": "0.0.0",
           "scripts": {
             "ng": "ng",
             "start": "ng serve",
             "build": "ng build --configuration production",
             "watch": "ng build --watch --configuration develop",
             "test": "ng test",
             "cypress": "npx cypress open",
             "cypress:run": "npx cypress run"
           },
           "prettier": {
             "printWidth": 100,
             "singleQuote": true,
             "overrides": [
               {
                 "files": "*.html"
               }
             ]
           }
         }
```

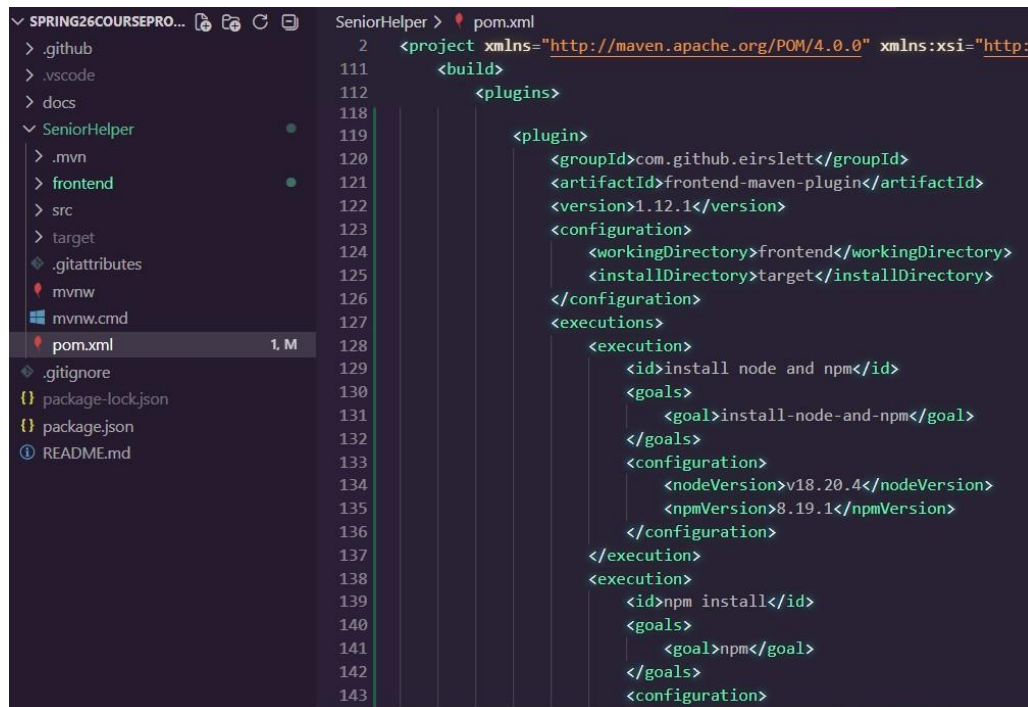
Step 3. Update Angular Production Environment Settings by adding a new file (environment.prod.ts) and setting it up so production points to the Sprint Boot server on port 8080.



```
SPRING26COURSEPROJECT
└─ SeniorHelper
   └─ frontend
      ├── .angular
      ├── cypress
      ├── dist
      ├── node_modules
      ├── public
      ├── src
      ├── .editorconfig
      ├── .gitignore
      ├── angular.json
      ├── cypress.config.ts
      ├── environment.prod.ts
      └── package-lock.json

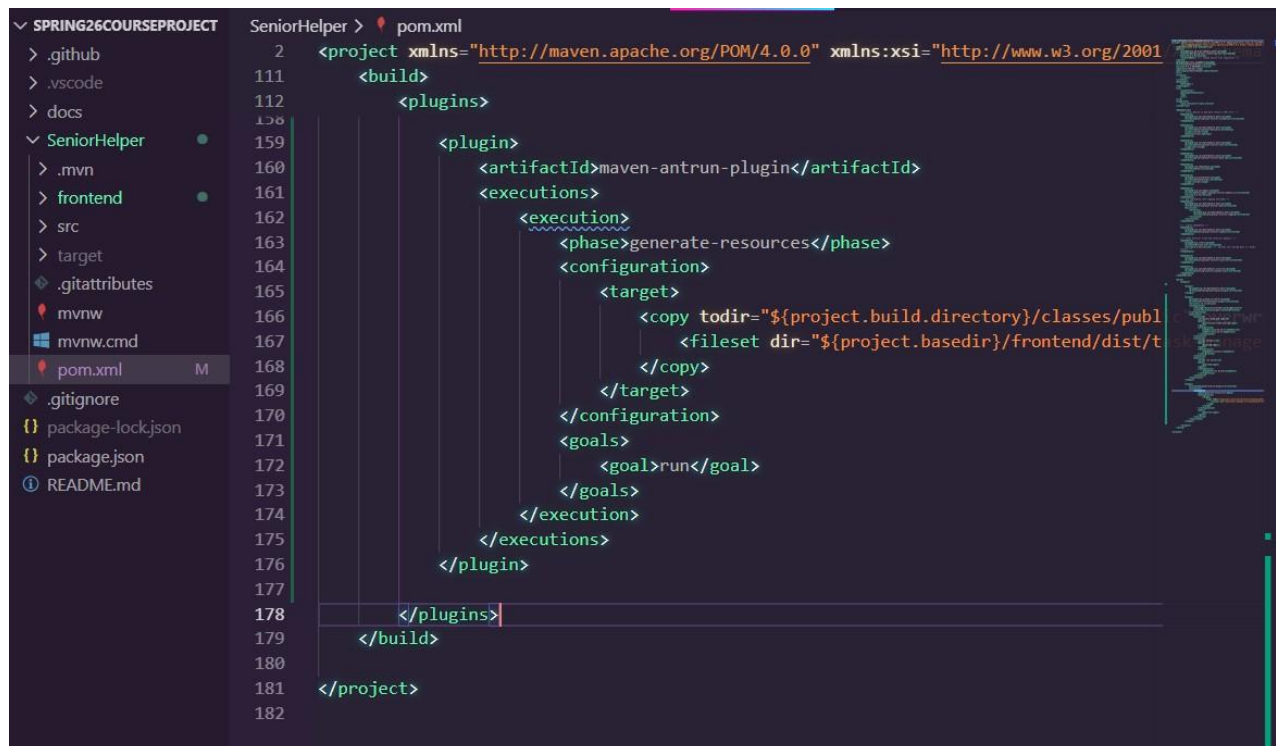
SeniorHelper > frontend > TS environment.prod.ts > ...
1 export const environment = {
2   production: true,
3   apiUrl: "http://localhost:8080",
4   authURL: "http://localhost:8080"
5 };
```

Step 4. Add the frontend-maven-plugin inside the <build><plugins> section in the pom.xml file.



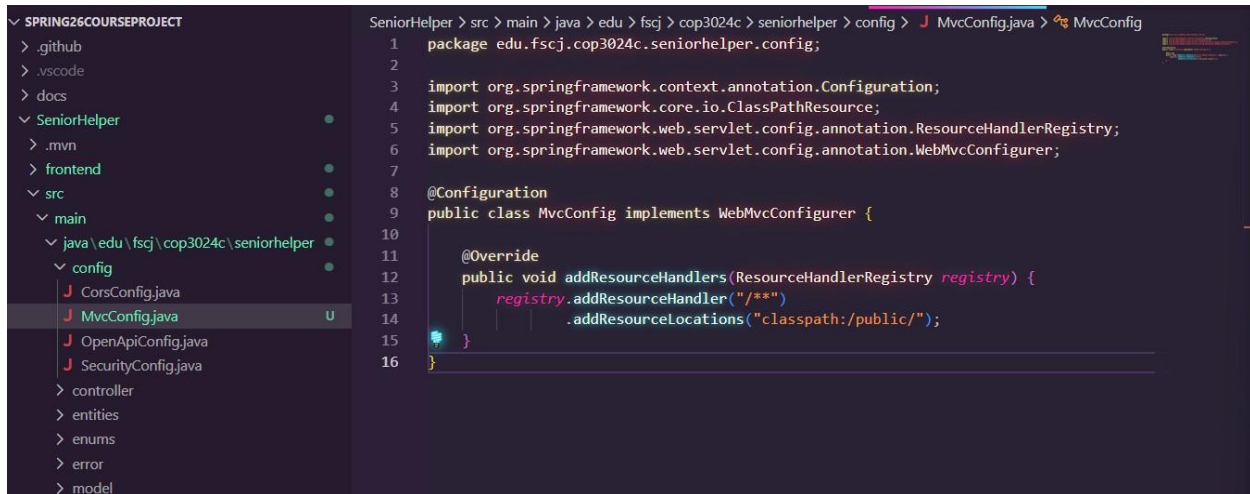
```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
111 <build>
112 <plugins>
118 <plugin>
119 <groupId>com.github.eirslett</groupId>
120 <artifactId>frontend-maven-plugin</artifactId>
121 <version>1.12.1</version>
122 <configuration>
123 <workingDirectory>frontend</workingDirectory>
124 <installDirectory>target</installDirectory>
125 </configuration>
126 <executions>
127 <execution>
128 <id>install node and npm</id>
129 <goals>
130 <goal>install-node-and-npm</goal>
131 </goals>
132 <configuration>
133 <nodeVersion>v18.20.4</nodeVersion>
134 <npmVersion>8.19.1</npmVersion>
135 </configuration>
136 </execution>
137 <execution>
138 <id>npm install</id>
139 <goals>
140 <goal>npm</goal>
141 </goals>
142 <configuration>
```

Step 5. Add maven-antrun-plugin to copy Angular build files into Spring Boot

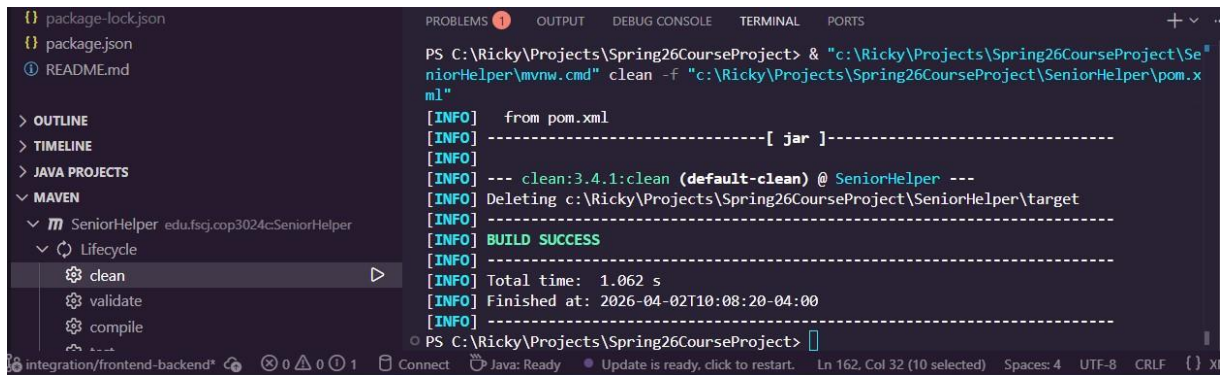


```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
111 <build>
112 <plugins>
159 <plugin>
160 <artifactId>maven-antrun-plugin</artifactId>
161 <executions>
162 <execution>
163 <phase>generate-resources</phase>
164 <configuration>
165 <target>
166 <copy todir="${project.build.directory}/classes/public" overwrite="true">
167 <fileset dir="${project.basedir}/frontend/dist" />
168 </copy>
169 </target>
170 </configuration>
171 <goals>
172 <goal>run</goal>
173 </goals>
174 </execution>
175 </executions>
176 </plugin>
177
178 </plugins>
179 </build>
180
181 </project>
```

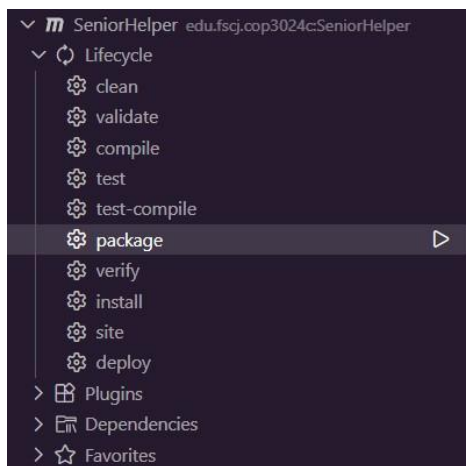
Step 6. Add Spring MVC configuration to serve Angular



Step 7. Use the Maven tool to run “clean”



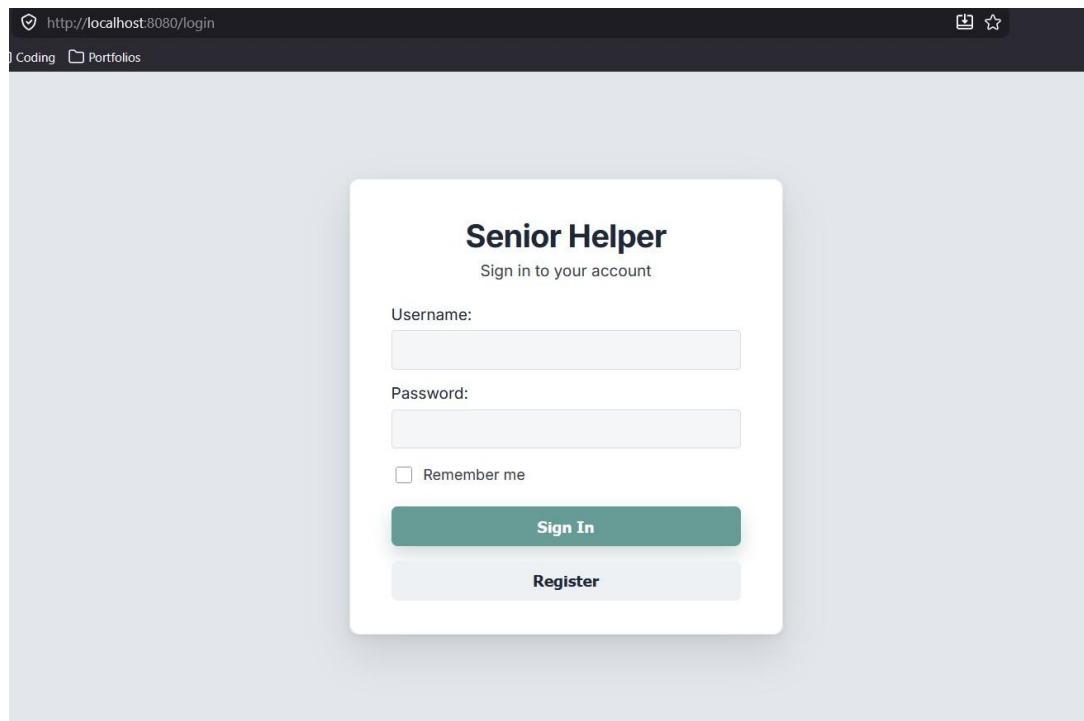
Step 8. Use the Maven tool “package”



Step 9. Run the resulting JAR file.

[illegible]

Application is now successfully combined and is running through the browser on port 8080.



© 2026 Florida State College at Jacksonville. All Rights Reserved.

Build Process Flow

1. Developer runs mvn clean package
2. Maven cleans previous build (target directory removed)
3. frontend-maven-plugin installs Node.js and npm
4. frontend-maven-plugin runs npm install
5. frontend-maven-plugin runs npm run build (Angular production build)
6. Angular build output generated in frontend/dist/SeniorHelperFrontEnd/browser
7. maven-antrun-plugin copies Angular build files into target/classes/public
8. Spring Boot Maven plugin packages backend and static frontend files into executable JAR
9. JAR file created in target/SeniorHelper-0.0.1-SNAPSHOT.jar
10. User runs java -jar SeniorHelper-0.0.1-SNAPSHOT.jar
11. Spring Boot application starts embedded server
12. Application served at http://localhost:8080